Exploring Multi-Agents for Software Vulnerability Remediation

Saviour Owolabi*, Jessica Pourleyli†, Ahmad Abdellatif*, Glaucia Melo†
* *Department of Electrical and Software Engineering*
University of Calgary
Calgary, Canada
{saviour.owolabi, ahmad.abdellatif}@ucalgary.ca
† *Department of Computer Science*
Toronto Metropolitan University
Toronto, Canada
jessica.pourleyli@torontomu.ca, glaucia@torontomu.ca

***Abstract*—Automated vulnerability remediation is a central challenge in software security, given its potential to reduce the risk of vulnerability exploitation. Large Language Models (LLMs), which have advanced the state-of-the-art across numerous software engineering tasks, have naturally been applied to this challenge. Motivated by the limitations of simple prompting and single-agent systems, we propose and evaluate a four-agent vulnerability remediation system (*Supervisor Agent*, *Investigator Agent*, *Patch Agent*, and *Verification Agent*) on 70 verified real-world vulnerabilities from the Python subset of PATCHEVAL. Relative to a single-agent baseline, the multi-agent system achieves up to 50% overall remediation success (passing both security and unit tests), compared with 48.6% for the baseline. An ablation study of the component agents shows that removing the *Patch Agent* produces the largest performance drop, reducing security-test success by 28.9% and overall remediation success by 37.1%. This result suggests that a dedicated patching agent is the most important contributor to overall multi-agent performance. Furthermore, analyzing success by patch complexity, measured by the number of hunks, shows that the multi-agent system is more effective than the single-agent baseline on more complex patches.**

I. Introduction

Many software systems continue to operate with known vulnerabilities long after their disclosure [1]. For example, nearly eighteen months after Log4Shell (a vulnerability associated with average incident-response costs exceeding US$90,000) was disclosed, it remained the second-most commonly exploited vulnerability [2], [3]. Consequently, researchers have devoted significant effort to developing approaches for automatic vulnerability detection and remediation [4]–[7].

Prior work shows that LLMs achieve promising performance on various software engineering tasks, including code generation [8], program analysis [9], and vulnerability detection [10]. In vulnerability remediation, LLMs are particularly appealing because they can generate candidate patches rapidly, reducing the time between vulnerability disclosure and remediation and the risk of exploitation in that period. LLM-based automated remediation also has the potential to reduce the burden on developers and maintainers, especially in open-source ecosystems where maintenance resources are often limited [11].

However, several documented limitations hinder the realization of these benefits [12]–[14]. Prompt-only LLM-driven repair can be sensitive to prompt design, produce implausible or incorrect patches, and require careful selection of source-code context [15], [16]. Single-agent systems seek to overcome these challenges by equipping LLMs with external tools for repository inspection, code modification, and test execution, enabling them to carry out autonomous, multi-step repair workflows [14]. PATCHAGENT, for example, integrates fault localization, patch generation, and validation within a single agent using a language server, a patch verifier, and task-specific interaction strategies, achieving strong results [6]. Despite this progress, concentrating vulnerability localization, tool use, patch generation, and validation within one agent creates a substantial coordination burden. Existing evaluations show that single-agent systems frequently fail to locate vulnerable code, use analysis tools effectively, or generate complete and valid patches, particularly when vulnerability information is limited [12], [14].

Multi-agent systems offer an alternative by distributing these responsibilities across specialized agents and have shown promise in a range of software engineering tasks [17], [18]. In vulnerability remediation, assigning analysis, patch generation, and validation to separate agents may reduce the burden on each agent and improve overall system performance [7], [19]. Nevertheless, it remains unclear whether multi-agent remediation systems consistently outperform comparable single-agent baselines [7], [12]. The sources of any observed performance gains are also poorly understood because existing evaluations rarely isolate the contributions of individual agents or specific agent-design choices [7].

To address these gaps, we propose and evaluate a four-agent vulnerability remediation system that reflects the core security-patching workflow: understanding each vulnerability within its repository context, planning a fix, generating a patch, and verifying both exploit removal and functional correctness. Specifically, the proposed system consists of a *Supervisor*

Agent for orchestration and planning, an *Investigator Agent* for gathering repository and vulnerability context, a *Patch Agent* for generating patches, and a *Verification Agent* for running tests and generating feedback when a patch attempt fails.

We evaluate the multi-agent system on 70 real-world Python vulnerabilities drawn from the PATCHEVAL benchmark [5]. Each evaluated PATCHEVAL instance includes security and unit tests, allowing us to assess whether generated patches resolve the target vulnerability while preserving the functionality covered by the unit tests. We use a tool-equivalent single-agent system as the baseline. Our study is guided by the following research questions:

- **RQ1: How Effective Is Our Multi-Agent Approach Compared with a Single-Agent System for Vulnerability Remediation?**
  We examine whether a multi-agent system that assigns specialized roles across the vulnerability remediation process improves performance compared with assigning the same task to a single agent. We measure successful remediation using security tests and assess functionality preservation using the unit tests provided by PATCHEVAL. Our evaluation across three state-of-the-art LLMs (GPT-4.1, Gemini-2.5-Pro, and DeepSeek-V3.2) shows that, on average, the multi-agent system increases the proportion of successful remediations (patches that pass both the security and unit tests) by 9.6% relative to the baseline (38.1% vs. 34.8%).
- **RQ2: What Aspects of the Multi-Agent System Contribute Most to Its Success?**
  Building on the findings from RQ1, we examine which agents contribute most to the multi-agent system's performance. To answer this question, we conduct an ablation study that isolates each agent's contribution on the vulnerabilities successfully patched by the best-performing multi-agent system. We find that removing the *Patch Agent* produces the largest drop in overall success, yielding 37.1% fewer successful remediations than the full system, compared with 28.6% fewer when the *Investigator Agent* is removed and 25.7% fewer when the *Verification Agent* is removed. These results suggest that separating patch generation into a dedicated agent is critical to the multi-agent system's overall effectiveness.

In addition to addressing the research questions above, we examine how patch complexity affects the performance of our proposed approach. We use the number of hunks in the ground-truth patch as a proxy for complexity and compare our approach with the baseline across patches of varying complexity. Our results indicate that the multi-agent system is better able to handle complex repairs. For example, the multi-agent system successfully remediates up to 36.8% of vulnerabilities requiring patches with three or more hunks, compared with 26.3% for the single-agent baseline. Our paper makes the following contributions:

- To the best of our knowledge, our work provides the first systematic evaluation of a multi-agent vulnerability remediation system, including comparisons with a single-agent baseline.
- We conduct an ablation study to isolate and quantify each agent's contribution to the system's remediation performance.
- We make available a replication package [20] containing our implementation of the aforementioned architecture, which supports further research in this area.

Paper Organization. The remainder of this paper is organized as follows: Section II reviews related work. Section III describes the architecture of the proposed system. Section IV presents the dataset and evaluation framework. Section V reports the experimental results, followed by a discussion in Section VI. Section VII outlines the threats to validity of our study, and Section VIII summarizes the findings and outlines directions for future work.

II. Related Work

We examine prior work related to LLM-assisted vulnerability remediation across three areas: *fault localization*, *vulnerability detection and repair*, and *multi-agent frameworks for software engineering*. Together, these research directions establish the foundations for automated vulnerability remediation systems by addressing where vulnerabilities occur, how they can be repaired, and how complex remediation workflows can be coordinated through specialized agents.

A. Fault Localization

Traditional fault localization methods, whether static or spectrum-based, often fail to capture the deeper semantics of code [21]. Recent studies demonstrate how LLMs can leverage contextual reasoning to locate faults more accurately, frequently outperforming classical methods [22], [23]. Some approaches, such as FlexFL, enhance precision by integrating LLM reasoning with traditional fault localization techniques [24]. Other approaches like LLM4FL introduce ordered context analysis and reflection-based re-ranking to handle large or multi-file projects [25]. Together, these studies show that iterative reasoning and deeper semantic understanding can enhance localization accuracy. This insight lays the groundwork for LLM-based debugging workflows.

B. Vulnerability Detection and Repair

In addition to fault localization, prior research has explored the use of LLMs for secure patch generation and vulnerability remediation. PATCHAGENT [6] proposes a repair workflow inspired by human security experts, combining vulnerability understanding, patch generation, and iterative validation. Similarly, AutoPatch [7] employs a multi-agent architecture to coordinate vulnerability analysis and patch generation for vulnerabilities in LLM-generated code. Other studies have explored fine-tuning code language models to generate candidate security patches [26]. Exploit-based evaluations of multi-step repair workflows show that iterative vulnerability repair can improve vulnerability detection and, in some cases, repair success, although overall gains remain modest [12]. Complementing these repair evaluations, SecureVibeBench assesses

whether code agents generate functionally correct and secure code, using both functionality tests and security checks [27]. Additionally, fine-tuning LLMs for context-aware vulnerability detection across multiple CWE categories has been shown to outperform traditional static analysis approaches [28].

C. Multi-Agent Frameworks for Software Engineering

While multi-agent systems have demonstrated promise across a wide range of software engineering tasks, it remains unclear when role-specialized multi-agent architectures provide advantages over strong single-agent alternatives for vulnerability remediation. This question is particularly important because vulnerability remediation requires balancing multiple objectives, including vulnerability removal, functionality preservation and the effective use of security-relevant contextual information.

Recent multi-agent frameworks have demonstrated effectiveness in coordinating LLM agents for software development. ChatDev [19] and MetaGPT [17] employ structured, role-based coordination mechanisms to support multi-agent software development, while AutoGen [29] provides a more flexible, conversation-driven framework that supports both static and dynamic agent configurations. These frameworks have shown success in code generation, requirements engineering, and test synthesis tasks.

More recently, agentic approaches have been applied to security-focused tasks. PATCHAGENT [6] proposes a repair workflow inspired by human security experts that combines vulnerability understanding, patch generation, and iterative validation, while AutoPatch [7] employs a multi-agent architecture to coordinate vulnerability analysis and patch generation for vulnerabilities created by LLMs during code generation. These studies demonstrate the promise of agentic workflows for vulnerability remediation and suggest that task decomposition can improve repair effectiveness.

However, existing studies have primarily focused on remediation effectiveness and provided limited insight into the benefits of multi-agent specialization relative to tool-equivalent single-agent approaches. Furthermore, evaluations rarely consider both vulnerability removal and functionality preservation while simultaneously accounting for security and unit testing.

Our work aims to address these gaps by directly comparing single-agent and multi-agent remediation systems on real-world vulnerabilities aggregated by the PATCHEVAL benchmark, while examining the role of explicit task specialization for vulnerability investigation, patch generation, and verification when used with the same toolsets.

III. System Architecture

The goal of this study is to improve the vulnerability remediation process using a multi-agent system. Our approach consists of four agents: the *Investigator Agent*, the *Patch Agent*, the *Verification Agent*, and the *Supervisor Agent*. The system receives as input a vulnerability description and its location in the target repository (file paths and line numbers), both provided by the PATCHEVAL benchmark. The agents then work together to produce a patch, a modification to the code aimed at remediating the vulnerability.

Figure 1 presents an overview of our approach. The *Investigator Agent* gathers repository and vulnerability context and synthesizes it into a report. The *Patch Agent* uses this report to generate a candidate patch, while the *Verification Agent* runs the available tests and interprets their output. The *Supervisor Agent* coordinates communication among the specialist agents, decides which agent to invoke next, and determines whether to continue iterating or return the final patch. We describe each agent in detail next.

- **Investigator Agent.** Vulnerability remediation depends on the repository-specific context. A code pattern that is exploitable in one repository may be harmless in another, depending on how data enters the system, which checks are already in place, and how the affected code is used. Furthermore, changing a vulnerable function requires understanding its purpose, callers, inputs, outputs, side effects, and related utilities so that the patch removes the vulnerability without introducing regressions. The *Investigator Agent* is responsible for building this repository-specific view of the vulnerability before attempting a patch.
  To establish this context, the *Investigator Agent* uses two repository inspection tools: *ReadFile* and *FindClass*. The *ReadFile* tool provides file access, allowing the agent to inspect files (e.g., the file with the vulnerable code pattern as provided by the PATCHEVAL benchmark) or targeted line ranges around the vulnerable code. The *FindClass* tool accepts a symbol name and locates its definitions and usages, helping the agent identify related classes and functions along the vulnerable code path.
  The *Investigator Agent* then uses the information gathered with these tools to provide the *Supervisor Agent* with a report on repository-specific constraints that the patch must preserve. This includes API contracts, data-flow assumptions, existing security checks, coding conventions, and edge cases. If an initial patch attempt fails, the *Supervisor Agent* can ask the *Investigator Agent* to interpret verification failures and identify missing context that may explain why the prior patch failed.
- **Patch Agent.** This agent is responsible for making changes to the code to address the vulnerability (producing a patch). It receives a task-specific instruction from the *Supervisor Agent* that synthesizes the relevant repository context and constraints from the *Investigator Agent*'s report. This instruction is accompanied by PATCHEVAL metadata for the current case, including the vulnerability description, affected file and code snippet, and relevant CWE identifiers. The *Patch Agent* translates this instruction into a concrete code edit that removes the vulnerability while preserving expected behaviour. If an initial patch fails validation, the Verification Agent returns its findings to the *Supervisor Agent*, which incorporates the relevant feedback into a revised instruction for the *Patch Agent*.
  The *Patch Agent*'s system prompt instructs the LLM to

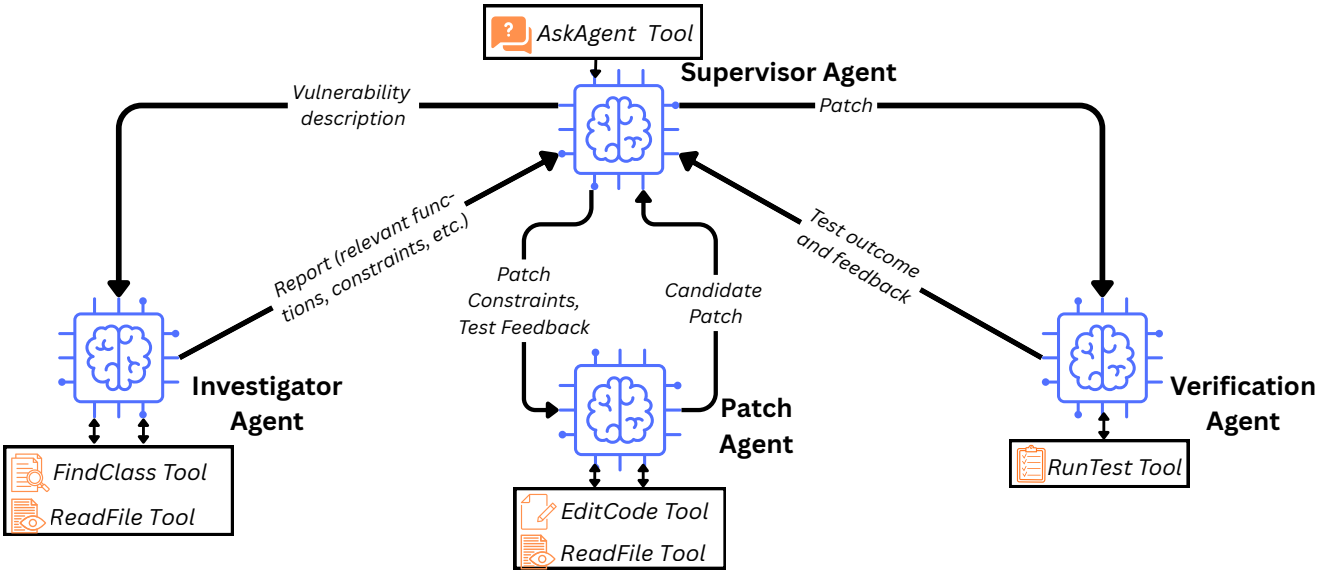


Fig. 1: Multi-agent vulnerability remediation workflow

implement a minimal and complete change that modifies only the vulnerable files and preserves expected behaviour. To support this process, the *Patch Agent* has access to the *ReadFile* and *EditCode* tools. Before making an edit, it uses *ReadFile* to inspect the current source file to determine what changes to make. It then generates replacement code blocks and passes them to *EditCode*. This tool locates the original block, applies the replacement, verifies that the change was written, and records the resulting patch.

After applying the edit, the *Patch Agent* returns an implementation summary and the edit outcome to the Supervisor Agent. The summary identifies the modified file and explains how the change addresses the vulnerability. The *Supervisor Agent* subsequently calls the *Verification Agent* to evaluate the patch. If an initial patch fails validation, the *Supervisor Agent* incorporates the Verification Agent's findings into a revised instruction for the Patch Agent.

- **Verification Agent.** Vulnerability patches must satisfy two requirements: they must remove the exploitable behaviour and preserve the program's expected functionality. A patch that looks plausible may still fail to remediate a vulnerability or may introduce regressions. The *Verification Agent* is responsible for validating that a patch satisfies both requirements. It receives as input the patch produced by the *Patch Agent*, runs the available validation tests, and reports whether the patch fixes the vulnerability.

  To perform its role, the *Verification Agent* has access to a *RunTest* tool. The *RunTest* tool executes the PATCHEVAL validation tests, including the security test and the unit tests (when available). The tool also captures and returns the test outputs, including the pass/fail status and any standard output produced during execution.

  The *Verification Agent* uses this output to determine whether the patch fixed the vulnerability and whether it preserved existing behaviour. If validation fails, it returns the observed failure mode, relevant output excerpts, and concrete suggestions to the *Supervisor Agent* for the next iteration.
- **Supervisor Agent.** The *Supervisor Agent* coordinates all agents involved in the remediation task. Its system prompt identifies the available specialists (the Investigator, Patch, and Verification Agents described above) and defines their respective responsibilities. At the beginning of each run, the *Supervisor Agent* is also provided with the vulnerability description, affected file and code snippet, and relevant CWE identifiers from PATCHEVAL.

  The prompt defines the initial remediation workflow. The *Supervisor Agent* first asks the *Investigator Agent* to analyze the vulnerable code and uses the resulting report to develop a repair strategy. It then asks the *Patch Agent* to implement this strategy and subsequently requests the Verification Agent to evaluate the resulting patch.

  At each stage of the workflow, the *Supervisor Agent* selects the next specialist based on the current remediation phase and the information returned by previous agent calls. It invokes the selected specialist through the *AskAgent* tool using a task-specific instruction and receives the specialist's response. This response is incorporated into the Supervisor Agent's context and used to guide subsequent decisions. The *Supervisor Agent* can also use the *ReadFile* tool to inspect changes introduced by the Patch Agent.

  If the initial patch fails verification, the Verification Agent's response is returned to the *Supervisor Agent* as feedback for the next iteration. The *Supervisor Agent* uses this feedback to determine the appropriate next action. A failure caused by a missing repository context may prompt the *Supervisor Agent* to re-involve the *Investigator Agent*, whereas a specific implementation issue may prompt it to provide revised instructions to the *Patch Agent* before requesting another

verification. Based on the outcomes of these iterations, the *Supervisor Agent* determines whether remediation is complete or requires further refinement.

IV. Evaluation Framework

This section describes the dataset and experimental setup used to evaluate the proposed multi-agent system.

A. Baseline

Wei et al. [5] evaluate three general-purpose agents on the PATCHEVAL benchmark: SWE-Agent [30], OpenHands [31], and Claude Code [32]. We include their published results for completeness and broader comparison. However, these systems use different prompts and tools from our proposed approach. Their results, therefore, cannot isolate the effect of the multi-agent design.

Our primary baseline is therefore a single-agent version of the proposed system. This agent combines the investigation, planning, patching, and verification responsibilities assigned to separate agents in the multi-agent system. It receives the same vulnerability inputs and operates in the same repository and PATCHEVAL environment. Its system prompt directs it to investigate the vulnerability, formulate a repair strategy, inspect and modify the affected code, run the security and unit tests, and revise the patch when verification fails. It also has direct access to the same tools as the multi-agent system, described in Section III: *ReadFile*, *FindClass*, *EditCode*, and *RunTest*.

B. Dataset

We evaluate the proposed multi-agent remediation system using the PATCHEVAL benchmark [5]. PATCHEVAL is a recently introduced benchmark of real-world vulnerabilities that has been used to assess agentic vulnerability repair systems [33], [34]. It provides 230 Docker-based evaluation environments [35] spanning projects written in Go (83), JavaScript (77), and Python (70). Each environment includes security tests derived from proof-of-concept exploits, as well as unit tests. Together, these tests determine whether a generated patch removes the vulnerability while preserving expected functionality.

Our evaluation covers all 70 executable Python instances, representing 70 distinct Common Vulnerabilities and Exposures (CVEs) across 48 repositories. We restrict our study to this subset to keep the cost of evaluating three LLMs and conducting the ablation study manageable. In total, our evaluation comprises 534 distinct repair runs. Focusing on a single language also reduces environmental variability and makes our study easier to replicate.

TABLE I: Features of our dataset.

Metric	Min	Median	Max
Lines of Code Changed	1	12	123
Hunks	1	2	6
Number of Files	1	1	3

TABLE II: Distribution of vulnerabilities by CWE.

CWE ID	Weakness Name	Count
CWE-73	External Control of File Name or Path	15
CWE-22	Path Traversal	15
CWE-94	Improper Control of Generation of Code	11
CWE-77	Command Injection	11
CWE-78	Improper Neutralization of Special Elements used in an OS Command	11

As shown in Table I, the vulnerabilities in our dataset vary in complexity, measured by the number of hunks in the ground-truth patch, as well as the number of files and lines of code they modify. They also span 28 Common Weakness Enumeration (CWE) categories, with the five most common of these shown in Table II.

C. Large Language Models

To ensure the generalizability of our approach, we evaluate the proposed multi-agent remediation approach with three state-of-the-art LLMs:

- **GPT-4.1**. We use GPT-4.1 as the primary model because it has been reported to improve on prior OpenAI models in coding, long-context use, and agentic software-engineering tasks [36].
- **Gemini-2.5-Pro**. We include Gemini-2.5-Pro as a proprietary frontier model with strong reasoning, coding, and long-context capabilities [37].
- **DeepSeek-V3.2**. We include DeepSeek-V3.2, an open-weight model, to assess whether the observed effects of agent specialization generalize beyond proprietary LLM services [38].

We selected these models for three reasons. First, each has demonstrated strong performance on software engineering or code-oriented tasks [36]–[38]. Second, all three were evaluated by Wei et al. [5], enabling a direct comparison between our approach and their reported baseline results. Third, the models' documented knowledge cutoffs precede the release of the PATCHEVAL benchmark, reducing the risk of benchmark contamination through exposure during pretraining.

Prompting. We use two types of prompts: fixed system prompts and task-specific prompts. System prompts define the roles, responsibilities, and behaviour of each agent, with separate prompts for the Supervisor, Investigator, Patch, and Verification Agents. In contrast, task prompts provide information specific to a remediation task, including the PATCHEVAL vulnerability description, details about affected files and lines, and CWE information when available.

D. Metrics

To assess the performance of our multi-agent system, we use the following metrics:

1. Security Test Success: We utilize the security tests (proof-of-concept exploits) provided by PATCHEVAL to determine if a patch effectively addresses the vulnerability. For each approach, we measure the number and proportion of cases that pass these security tests.

2. Overall Success: A patch achieves overall success if it passes both the security tests and the PATCHEVAL unit tests, indicating that it addresses the target vulnerability while preserving the functionality covered by the unit tests. For each approach, we measure the number and proportion of cases that satisfy both criteria.

E. Experimental Setup

We used a fixed system prompt for each agent in the multi-agent system and a separate fixed system prompt for the single-agent baseline; these prompts remained unchanged across samples. To ensure consistency across runs, each execution was performed with the underlying LLM's temperature set to 0 and was subject to a 300K-token cap for cost feasibility. We passed the same input data to each approach: relevant metadata from the PATCHEVAL benchmark, including the vulnerability description and the location of the vulnerable code segment. After each approach generated a candidate fix, we extracted the resulting patch and evaluated it in the environment provided by PATCHEVAL.

V. Results

In this section, we present the findings of our study regarding our RQs. For each RQ, we present the motivation for the question, describe the approach used to answer it, and discuss the results of our analysis.

A. RQ1: How Effective Is Our Multi-Agent Approach Compared with a Single-Agent System for Vulnerability Remediation?

Motivation. Multi-agent systems introduce additional design complexity compared with single-agent systems [39]. It is therefore important to determine whether role specialization improves remediation effectiveness and to quantify the magnitude of any improvement. Such evidence can support informed decisions about whether the benefits of a multi-agent design justify its added complexity.

Approach. To answer this RQ, we evaluate our multi-agent system described in Section III on the 70 Python vulnerabilities drawn from PATCHEVAL. To put results into context, we evaluate the baseline system using the same dataset. We assess each generated patch from our approach and baseline along two dimensions: whether it remediates the vulnerability, measured by passing the security tests, and whether it preserves existing functionality, measured by passing the PATCHEVAL unit tests. For each system, we vary the underlying LLM across GPT-4.1, Gemini-2.5-Pro, and DeepSeek-V3.2 to examine whether the observed patterns are consistent across models.

Results. Table III reports the results of our evaluation of the multi-agent system and baseline across all evaluated LLMs, alongside the performance of the general-purpose agents reported by Wei et al. [5]. Each row represents an underlying LLM, and the table reports baseline and multi-agent results for two criteria: security-test success and overall success, where overall success requires passing both the security tests and the PATCHEVAL unit tests.

As shown in Table III, the multi-agent system outperforms the baseline on average across the three evaluated models. In particular, average security-test success increases by 4.3 percentage points, from 24.7 patches (35.2%) to 27.7 patches (39.5%). In addition, average *overall success* increases by 3.3 percentage points, from 24.3 patches (34.8%) for the baseline to 26.7 patches (38.1%) for the multi-agent system.

This average improvement is reflected in two of the three individual model results. For GPT-4.1, security-test success increases by 4.3 percentage points, from 50% to 54.3%, while overall success increases by 1.4 percentage points, from 48.6% to 50.0%. Gemini-2.5-Pro follows the same pattern, with both security-test success and overall success increasing from 8.6% to 18.6%. In contrast, for DeepSeek-V3.2, the single-agent baseline slightly outperforms the multi-agent system under both criteria (47.1% vs. 45.7%).

In the GPT-4.1 runs, two cases show how agents with different roles helped the multi-agent system recover from failed patch attempts, while the corresponding single-agent runs failed validation. For CVE-2017-0360, a flaw in Tryton's file-access checks [40], [41], the single agent normalized the paths but kept the unsafe string-prefix check and reported success even though the security test failed. The multi-agent system's first patch also failed because it was incompatible with the target environment, but the *Verification Agent* rejected it and the *Supervisor Agent* requested a compatible path-boundary check that passed validation. For CVE-2022-0577, a flaw in Scrapy's cross-origin redirect handling [42], [43], the single agent recognized that cookies could be added back but changed only part of the redirect flow and eventually concluded that it could not fix the problem locally. The multi-agent system also missed this behavior initially, but the *Supervisor Agent* asked the *Investigator Agent* to inspect Scrapy's cookie handling. This investigation showed where cookies were added back, allowing the *Patch Agent* to update both redirect paths and pass validation. In both cases, the multi-agent system used failed tests to decide what to investigate and change next.

These results suggest that role specialization in a multi-agent system can improve performance in vulnerability remediation. Across the evaluated models, the multi-agent system generally outperforms the baseline and achieves higher average performance under both evaluation criteria. For DeepSeek-V3.2, where the baseline performs slightly better, the difference is small.

> ***Across the three evaluated models, the multi-agent system achieves an average overall success of 26.7 patches (38.1%), compared with 24.3 patches (34.8%) for the baseline, a 9.6% improvement.***

B. RQ2: What Aspects of the Multi-Agent System Contribute Most to Its Success?

Motivation. RQ1 demonstrates that the multi-agent system outperforms the baseline. However, this improvement may come at the cost of greater system complexity [39]. RQ2, therefore, examines the contributions of each agent in our

TABLE III: Remediation performance for single- and multi-agent systems across evaluated LLMs, alongside agent results reported by Wei et al. [5].

Model / Agent	**Security Test Success**		**Overall Success (Security & Unit Tests)**	
	Single-Agent	**Multi-Agent**	**Single-Agent**	**Multi-Agent**
GPT-4.1	35 (50%)	38 (54.3%)	34 (48.6%)	35 (50.0%)
Gemini-2.5-Pro	6 (8.6%)	13 (18.6%)	6 (8.6%)	13 (18.6%)
DeepSeek-V3.2	33 (47.1%)	32 (45.7%)	33 (47.1%)	32 (45.7%)
Average	**24.7 (35.2%)**	**27.7 (39.5%)**	**24.3 (34.8%)**	**26.7 (38.1%)**
Agents evaluated by PATCHEVAL [5]				
SWE-Agent [5], [30] (Gemini-2.5-Pro)	12 (17.1%)	–	8 (11.4%)	–
OpenHands [5], [31] (Gemini-2.5-Pro)	8 (11.4%)	–	7 (10.0%)	–
ClaudeCode [5], [32] (Gemini-2.5-Pro)	6 (8.6%)	–	5 (7.1%)	–

approach to the performance of the multi-agent system. These findings can help practitioners determine when the benefits of additional agents justify their overhead in vulnerability remediation systems.

Approach. To answer this question, we conduct a leave-one-specialist-out ablation study on the 38 vulnerabilities where the GPT-4.1-based multi-agent system passes the security tests. We restrict this evaluation to GPT-4.1 because it is the best-performing LLM evaluated in RQ1 and because using a single model keeps the computational cost of the ablation study manageable.

In each ablation, we remove one specialist agent (i.e., the *Investigator Agent*, *Patch Agent*, or *Verification Agent*) and revise the *Supervisor Agent*'s prompt and tool access so that it assumes the responsibilities of the removed specialist. For example, when ablating the *Verification Agent*, we give the *Supervisor Agent* access to the *RunTest* tool in addition to the *AskAgent* tool and extend its system prompt to include the verification responsibilities previously assigned to the *Verification Agent*. We hold the *Supervisor Agent* constant across all ablations because it provides the orchestration mechanism for invoking the specialist agents and managing the remediation workflow.

Results. Table IV compares the full multi-agent system with each ablation variant, reporting both security-test success and overall success. Drops are measured relative to the full system. As shown in the table, removing the *Patch Agent* causes the largest performance loss. Security-test success decreases by 11 cases, from 38 (100.0%) to 27 (71.1%). Overall success, which requires passing both the security and unit tests, similarly drops by 13 cases, from 35 (92.1%) to 22 (57.9%).

The other ablations also reduce performance, but to a lesser extent. Specifically, removing the *Verification Agent* reduces security-test success by 10 cases, while removing the *Investigator Agent* reduces it by 9 cases. For overall success, the Investigator and Verification ablations retain 25 and 26 repairs, respectively, compared with only 22 repairs when the *Patch Agent* is removed.

These results indicate that while all specialist agents contribute to repair performance, the *Patch Agent* is the most critical component. Separating patch generation into a dedicated agent appears especially important for preserving the full system's remediation success. One possible explanation is that patch generation is the central bottleneck in the repair pipeline. The *Investigator* and *Verification Agent*s provide useful support by identifying vulnerability causes and checking candidate repairs, but the *Patch Agent* is responsible for converting this information into concrete code modifications. When this responsibility is merged into the broader workflow, the system may produce less targeted patches, overlook implementation details, or introduce changes that fail regression tests. This would explain why removing the *Patch Agent* causes the largest drop in both security-test success and overall success.

Our ablation study shows that removing the Patch Agent leads to the largest performance drop, with 13 fewer overall successes relative to the full multi-agent system, compared with 10 fewer when removing the Investigator Agent and 9 fewer when removing the Verification Agent. These results suggest that patch generation benefits most from a dedicated agent.

VI. Discussion

In this section, we examine how remediation performance varies with patch complexity. We also discuss the token-use overhead introduced by multi-agent coordination and the broader implications of agent-based vulnerability remediation.

A. Impact of Patch Complexity on Performance

In their study, Wei et al. [5] report that patching effectiveness decreases as patch complexity increases. To assess whether this trend holds in our setting, we focus on the GPT-4.1 runs, which achieved the highest overall success in RQ1, and analyze overall success as a function of patch complexity (measured by the number of *hunks* in the PATCHEVAL ground truth patch). As shown in Table V, the multi-agent system achieves higher overall success on patches with two hunks (42.9% vs. 33.3%) and three or more hunks (36.8% vs. 26.3%), while it underperforms the single-agent baseline

TABLE IV: Ablation on the 38 vulnerabilities remediated by the GPT-4.1 multi-agent system.

System Variant	Security Test Success	Security Test Drop	Overall Success (Security & Unit Tests)	Overall Success Drop
Full Multi-Agent	38 (100.0%)	0	35 (92.1%)	0
W/O Verification Agent	28 (73.7%)	−10	26 (68.4%)	−9
W/O Investigator Agent	29 (76.3%)	−9	25 (65.8%)	−10
W/O Patch Agent	27 (71.1%)	−11	22 (57.9%)	−13

on one-hunk patches (63.3% vs. 73.3%). We hypothesize that role separation in the multi-agent system allows it to more effectively plan and follow through with implementing patches of higher complexity, where coordinating changes across multiple locations is necessary.

TABLE V: Overall remediation success by patch complexity for GPT-4.1.

Hunks	Vulnerabilities	Single-Agent Repairs	Multi-Agent Repairs
1	30	22 (73.3%)	19 (63.3%)
2	21	7 (33.3%)	9 (42.9%)
3+	19	5 (26.3%)	7 (36.8%)

The multi-agent system improves over the single-agent baseline on multi-hunk patches, while the single-agent baseline performs better on one-hunk patches.

B. The Multi-Agent Orchestration Overhead

Although the multi-agent system repairs more vulnerabilities than the single-agent baseline, this improvement comes at the cost of higher token use. Table VI reports average token use for the 70 Python CVEs. With GPT-4.1, the multi-agent system uses 141.6K tokens per CVE on average, compared with 43.6K for the single-agent baseline. The same pattern holds for Gemini-2.5-Pro, where average token use increases from 33.9K to 72.4K. These results show that multi-agent coordination incurs token overhead for all evaluated models, though the overhead size depends on the underlying LLM.

TABLE VI: Token-use overhead by system and model.

Model	Single-Agent Tokens/CVE (K)	Multi-Agent Tokens/CVE (K)
GPT-4.1	43.6	141.6
Gemini-2.5-Pro	33.9	72.4
DeepSeek v3.2	33.4	109.5

Determining which performance improvements justify this overhead is an important direction for future work. These results also motivate hybrid approaches that combine the strengths of both systems to capture most of the gains while reducing orchestration cost.

C. Implications

Beyond quantitative performance gains, our study illustrates a broader shift in software engineering practice, from prompt engineering toward agent design as a new layer of abstraction in LLM-driven development. Rather than optimizing isolated model instructions, researchers and practitioners architect collaborative systems of specialized agents that decompose reasoning tasks, interact through structured protocols, and jointly pursue debugging goals. This evolution mirrors historical progress in software design, where modularization and the separation of concerns have improved maintainability and scalability. Similarly, role-specialized agents embody modular reasoning, allowing each component to focus on a distinct cognitive function such as exploration, verification, or explanation.

VII. Limitations and Threats to Validity

Our findings provide evidence regarding the effectiveness of role-specialized multi-agent systems for vulnerability remediation. However, several factors may influence the interpretation and generalization of these results. In this section, we discuss some of these factors.

The primary goal of this study is to evaluate whether role-specialized multi-agent coordination can improve vulnerability remediation when compared to a single-agent approach. However, architectural differences are inherently coupled with differences in prompt structure, context organization and the flow of information that the agent receives. The multi-agent system distributes reasoning across specialized agents, each with distinct responsibilities and interaction histories, whereas the single-agent baseline orchestrates all these tasks within a unified context. Consequently, some of the observed performance differences may arise from how information is structured and presented rather than agent specialization alone. To mitigate this threat, both systems were provided access to the same tool selections, vulnerability information, and remediation objectives, and were designed to follow comparable remediation workflows. However, completely isolating the effects of architecture from prompting remains a challenge.

Our evaluation compares the proposed multi-agent system with a tool-equivalent single-agent baseline. This design supports a controlled comparison between assigning remediation responsibilities to one agent and distributing them across specialized agents, but it limits the scope of our conclusions. Specifically, the results do not establish that the proposed system outperforms other single-agent or multi-agent approaches that use different planning, reflection, memory, or tool-use strategies. Future work will compare the proposed system with a broader range of agentic vulnerability-remediation approaches.

Our evaluation uses a subset of the PATCHEVAL benchmark, comprising 70 verified real-world vulnerabilities spanning multiple repositories and CWE categories. While PATCHEVAL provides a realistic and standardized evaluation environment, the sample size remains modest and is limited to software projects developed in Python. Consequently, relatively small differences in the number of successful remediations can translate into noticeable percentage differences between systems. Furthermore, vulnerability distributions, repair strategies and remediation effectiveness may differ across programming languages and software ecosystems. Additional studies across larger datasets and multiple languages are necessary to establish the generality of the observed trends.

We limit each repair run to 300,000 tokens and set the temperature of each underlying LLM to 0 to control costs and ensure a consistent comparison between the two systems. Different token budgets or temperature settings may yield different results. Future work should examine the sensitivity of our findings to these parameters, and we encourage researchers to replicate the study under alternative configurations.

VIII. Conclusion

We presented a multi-agent system for automated vulnerability remediation that separates orchestration, vulnerability investigation, patch generation, and verification across a *Supervisor Agent*, *Investigator Agent*, *Patch Agent*, and *Verification Agent*. Across the three evaluated LLMs, the system improves average overall success by 9.6% relative to a tool-equivalent single-agent baseline (38.1% vs. 34.8%) and achieves up to 50.0% overall success, compared with 48.6% for the corresponding baseline. The ablation study further shows that the *Patch Agent* contributes most among the three evaluated specialist agents: removing it reduces security-test success by 28.9% and overall success by 37.1% relative to the full system, while removing the *Investigator Agent* or *Verification Agent* preserves more of the full system's successful repairs. Our analysis also shows that the benefit of multiple agents is not uniform across all vulnerabilities, with the multi-agent system outperforming the single-agent on patches with multiple hunks. Despite these encouraging results, there is room for further improvement. Future work should evaluate the system on broader benchmarks and additional programming languages. The differing performance patterns also motivate hybrid routing strategies that choose between single- and multi-agent workflows based on the vulnerability, expected patch complexity, and cost budget. Reducing multi-agent orchestration overhead remains important for making these systems practical and improving their adoption.

References

[1] C. Yang-Smith and A. Abdellatif, "Tracing vulnerabilities in maven: A study of cve lifecycles and dependency networks," in *2025 IEEE/ACM 22nd International Conference on Mining Software Repositories (MSR)*, 2025, pp. 349–353. [Online]. Available: https://ieeexplore.ieee.org/document/11025716

[2] "What is the Log4j Vulnerability? | IBM," Online Article, Jun. 2023, accessed: Jun. 25, 2026. [Online]. Available: https://www.ibm.com/think/topics/log4j

[3] Arctic Wolf Labs, "A Log4Shell (Log4j) retrospective," Dec. 2022. [Online]. Available: https://arcticwolf.com/resources/blog/log4j-retrospective/

[4] C. S. Xia, Y. Wei, and L. Zhang, "Automated Program Repair in the Era of Large Pre-trained Language Models," in *2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE)*, May 2023, pp. 1482–1494. [Online]. Available: https://ieeexplore.ieee.org/document/10172803

[5] Z. Wei, J. Zeng, M. Wen, Z. Yu, K. Cheng, Y. Zhu, J. Guo, S. Zhou, L. Yin, X. Su, and Z. Ma, "PATCHEVAL: A New Benchmark for Evaluating LLMs on Patching Real-World Vulnerabilities," Nov. 2025. [Online]. Available: https://arxiv.org/abs/2511.11019

[6] Z. Yu, Z. Guo, Y. Wu, J. Yu, M. Xu, D. Mu, Y. Chen, and X. Xing, "PATCHAGENT: A practical program repair agent mimicking human expertise," in *34th USENIX Security Symposium (USENIX Security 25)*. Seattle, WA: USENIX Association, Aug. 2025, pp. 4381–4400. [Online]. Available: https://www.usenix.org/conference/usenixsecurity25/presentation/yu-zheng

[7] M. Seo, W. Choi, M. You, and S. Shin, "Autopatch: Multi-agent framework for patching real-world cve vulnerabilities," arxiv preprint arxiv:2505.04195, 2025. [Online]. Available: https://arxiv.org/abs/2505.04195

[8] I. Ahmed, A. Aleti, H. Cai, A. Chatzigeorgiou, P. He, X. Hu, M. Pezzè, D. Poshyvanyk, and X. Xia, "Artificial intelligence for software engineering: The journey so far and the road ahead," *ACM Trans. Softw. Eng. Methodol.*, vol. 34, no. 5, May 2025. [Online]. Available: https://doi.org/10.1145/3719006

[9] P. Liu, C. Sun, Y. Zheng, X. Feng, C. Qin, Y. Wang, Z. Xu, Z. Li, P. Di, Y. Jiang, and L. Sun, "Llm-powered static binary taint analysis," *ACM Trans. Softw. Eng. Methodol.*, vol. 34, no. 3, Feb. 2025. [Online]. Available: https://doi.org/10.1145/3711816

[10] Y. Yang, B. Xu, X. Gao, and H. Sun, "Context-enhanced vulnerability detection based on large language models," *ACM Trans. Softw. Eng. Methodol.*, vol. 35, no. 8, Jul. 2026. [Online]. Available: https://doi.org/10.1145/3779222

[11] N. Kshetri, "Transforming cybersecurity with agentic ai to combat emerging cyber threats," *Telecommunications Policy*, vol. 49, no. 6, p. 102976, 2025. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S0308596125000734

[12] W. Wang, W. Ma, Q. Hu, Y. Zhang, J. Sun, B. Wu, Y. Liu, G. Xu, and L. Jiang, "Vulnrepaireval: An exploit-based evaluation framework for assessing large language model vulnerability repair capabilities," arxiv preprint arxiv:2509.03331, 2025. [Online]. Available: https://arxiv.org/abs/2509.03331

[13] L. Zhang, Q. Zou, A. Singhal, X. Sun, and P. Liu, "Evaluating large language models for real-world vulnerability repair in c/c++ code," in *Proceedings of the 10th ACM International Workshop on Security and Privacy Analytics*, ser. IWSPA '24. New York, NY, USA: Association for Computing Machinery, 2024, p. 49–58. [Online]. Available: https://doi.org/10.1145/3643651.3659892

[14] P. Wang, X. Liu, and C. Xiao, "CVE-bench: Benchmarking LLM-based software engineering agent's ability to repair real-world CVE vulnerabilities," in *Proceedings of the 2025 Conference of the Nations of the Americas Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*, L. Chiruzzo, A. Ritter, and L. Wang, Eds. Albuquerque, New Mexico: Association for Computational Linguistics, Apr. 2025, pp. 4207–4224. [Online]. Available: https://aclanthology.org/2025.naacl-long.212/

[15] H. Pearce, B. Tan, B. Ahmad, R. Karri, and B. Dolan-Gavitt, " Examining Zero-Shot Vulnerability Repair with Large Language Models ," in *2023 IEEE Symposium on Security and Privacy (SP)*. Los Alamitos, CA, USA: IEEE Computer Society, May 2023, pp. 2339–2356. [Online]. Available: https://doi.ieeecomputersociety.org/10.1109/SP46215.2023.10179420

[16] Y. Luo, B. Li, A. Singhal, P. Tseng, L. Zhang, Q. Zou, X. Sun, and P. Liu, "Exploring prompt patterns for effective vulnerability repair in real-world code by large language models," in *Proceedings of the 10th ACM International Workshop on Security and Privacy Analytics*, ser. IWSPA '25. New York, NY, USA: Association for Computing Machinery, 2025, p. 23–33. [Online]. Available: https://doi.org/10.1145/3716815.3729010

[17] S. Hong, M. Zhuge, J. Chen, X. Zheng, Y. Cheng, J. Wang, C. Zhang, z. wang, S. Yau, Z. Lin, L. Zhou, C. Ran, L. Xiao, C. Wu, and J. Schmidhuber, "Metagpt: Meta programming for a multi-agent

collaborative framework," in *International Conference on Learning Representations*, B. Kim, Y. Yue, S. Chaudhuri, K. Fragkiadaki, M. Khan, and Y. Sun, Eds., vol. 2024, 2024, pp. 23 247–23 275. [Online]. Available: https://proceedings.iclr.cc/paper_files/paper/2024/file/6507b115562bb0a305f1958ccc87355a-Paper-Conference.pdf

[18] J. He, C. Treude, and D. Lo, "Llm-based multi-agent systems for software engineering: Literature review, vision, and the road ahead," *ACM Trans. Softw. Eng. Methodol.*, vol. 34, no. 5, May 2025. [Online]. Available: https://doi.org/10.1145/3712003

[19] C. Qian, W. Liu, H. Liu, N. Chen, Y. Dang, J. Li, C. Yang, W. Chen, Y. Su, X. Cong, J. Xu, D. Li, Z. Liu, and M. Sun, "ChatDev: Communicative agents for software development," in *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, L.-W. Ku, A. Martins, and V. Srikumar, Eds. Bangkok, Thailand: Association for Computational Linguistics, Aug. 2024, pp. 15 174–15 186. [Online]. Available: https://aclanthology.org/2024.acl-long.810/

[20] S. Owolabi, J. Pourleyli, A. Abdellatif, and G. Melo, "Replication package for "exploring multi-agents for software vulnerability remediation"." [Online]. Available: https://github.com/Seyviour/vuln-agent

[21] R. Abreu, W. Mayer, M. Stumptner, and A. J. C. van Gemund, "Refining spectrum-based fault localization rankings," in *Proceedings of the 2009 ACM Symposium on Applied Computing*, ser. SAC '09. New York, NY, USA: Association for Computing Machinery, 2009, p. 409–414. [Online]. Available: https://doi.org/10.1145/1529282.1529374

[22] S. Kang, G. An, and S. Yoo, "A quantitative and qualitative evaluation of llm-based explainable fault localization," *Proc. ACM Softw. Eng.*, vol. 1, no. FSE, Jul. 2024. [Online]. Available: https://doi.org/10.1145/3660771

[23] A. Z. H. Yang, C. Le Goues, R. Martins, and V. Hellendoorn, "Large language models for test-free fault localization," in *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering*, ser. ICSE '24. New York, NY, USA: Association for Computing Machinery, 2024. [Online]. Available: https://doi.org/10.1145/3597503.3623342

[24] C. Xu, Z. Liu, X. Ren, G. Zhang, M. Liang, and D. Lo, "Flexfl: Flexible and effective fault localization with open-source large language models," *IEEE Trans. Softw. Eng.*, vol. 51, no. 5, p. 1455–1471, May 2025. [Online]. Available: https://doi.org/10.1109/TSE.2025.3553363

[25] M. N. Rafi, D. J. Kim, T.-H. Chen, and S. Wang, "A multi-agent approach to fault localization via graph-based retrieval and reflexion," arXiv preprint arXiv:2409.13642, 2025. [Online]. Available: https://arxiv.org/abs/2409.13642

[26] G. Bhandari, N. Gavric, and A. Shalaginov, "Generating vulnerability security fixes with code language models," *Information and Software Technology*, vol. 185, p. 107786, 2025. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S0950584925001259

[27] J. Chen, H. Huang, Y. Lyu, J. An, J. Shi, C. Yang, T. Zhang, H. Tian, Y. Li, Z. Li, X. Zhou, X. Hu, and D. Lo, "SecureVibeBench: Benchmarking secure vibe coding of AI agents via reconstructing vulnerability-introducing scenarios," in *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, M. Liakata, V. P. Moreira, J. Zhang, and D. Jurgens, Eds. San Diego, California, United States: Association for Computational Linguistics, Jul. 2026, pp. 24 144–24 168. [Online]. Available: https://aclanthology.org/2026.acl-long.1107/

[28] S. Wang, Y. Jiang, S. Xu, Y. Liu, M. Yin, and G. He, "Fine-tuning a vulnerability-specific large language model for a hybrid software vulnerability detection method," *Engineering Applications of Artificial Intelligence*, vol. 165, p. 113410, 2026. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S0952197625034414

[29] Q. Wu, G. Bansal, J. Zhang, Y. Wu, B. Li, E. E. Zhu, L. Jiang, X. Zhang, S. Zhang, A. Awadallah, R. W. White, D. Burger, and C. Wang, "Autogen: Enabling next-gen llm applications via multi-agent conversation," in *COLM 2024*, August 2024. [Online]. Available: https://www.microsoft.com/en-us/research/publication/autogen-enabling-next-gen-llm-applications-via-multi-agent-conversation-framework/

[30] J. Yang, C. Jimenez, A. Wettig, K. Lieret, S. Yao, K. Narasimhan, and O. Press, "Swe-agent: Agent-computer interfaces enable automated software engineering," in *Advances in Neural Information Processing Systems*, A. Globerson, L. Mackey, D. Belgrave, A. Fan, U. Paquet, J. Tomczak, and C. Zhang, Eds., vol. 37. Curran Associates, Inc., 2024, pp. 50 528–50 652. [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/2024/file/5a7c947568c1b1328ccc5230172e1e7c-Paper-Conference.pdf

[31] X. Wang, B. Li, Y. Song, F. F. Xu, X. Tang, M. Zhuge, J. Pan, Y. Song, B. Li, J. Singh, H. H. Tran, F. Li, R. Ma, M. Zheng, B. Qian, D. Shao, N. Muennighoff, Y. Zhang, B. Hui, J. Lin, R. Brennan, H. Peng, H. Ji, and G. Neubig, "Openhands: An open platform for AI software developers as generalist agents," in *The Thirteenth International Conference on Learning Representations*, 2025. [Online]. Available: https://openreview.net/forum?id=OJd3ayDDoF

[32] Anthropic, "Claude Code: Ai coding agent for terminal and ide," 2025. [Online]. Available: https://claude.com/product/claude-code

[33] S. Liu, L. Zhang, F. Liu, X. Lian, Y. Liu, and Y. Zhu, "Memrepair: Hierarchical memory for agentic repository-level vulnerability repair," arxiv preprint arxiv:2605.17444, 2026. [Online]. Available: https://arxiv.org/abs/2605.17444

[34] S. Cao, H. Ma, L. Yu, K. Ding, X. Liu, T. Y. Zhuo, B. Wang, X. Lin, X. Sun, L. Wang, and D. Lo, "Knowledge-enhanced agentic vulnerability repair," arxiv preprint arxiv:2607.00820, 2026. [Online]. Available: https://arxiv.org/abs/2607.00820

[35] Docker, Inc., "Docker Products." [Online]. Available: https://www.docker.com/products/

[36] OpenAI, "Introducing GPT-4.1 in the API," Apr. 2025, accessed: Jan. 18, 2026. [Online]. Available: https://openai.com/index/gpt-4-1/

[37] Google DeepMind, "Gemini 2.5 Pro: Model card," Jun. 2025. [Online]. Available: https://storage.googleapis.com/deepmind-media/Model-Cards/Gemini-2-5-Pro-Model-Card.pdf

[38] DeepSeek-AI *et al.*, "Deepseek-v3.2: Pushing the frontier of open large language models," arxiv preprint arxiv:2512.02556, 2025. [Online]. Available: https://arxiv.org/abs/2512.02556

[39] I. Radeva, I. Popchev, L. Doukovska, and M. Dimitrova, "Multi-agent coordination strategies vs. retrieval-augmented generation in llms: A comparative evaluation," *Electronics*, vol. 14, no. 24, 2025. [Online]. Available: https://www.mdpi.com/2079-9292/14/24/4883

[40] National Institute of Standards and Technology, "CVE-2017-0360 detail," National Vulnerability Database, accessed: Jul. 2, 2026. [Online]. Available: https://nvd.nist.gov/vuln/detail/CVE-2017-0360

[41] Tryton Foundation, "Tryton: Open source business software," [Online]. Available: https://www.tryton.org/, 2026.

[42] National Institute of Standards and Technology, "CVE-2022-0577 detail," National Vulnerability Database, accessed: Jul. 2, 2026. [Online]. Available: https://nvd.nist.gov/vuln/detail/CVE-2022-0577

[43] Scrapy contributors, "Scrapy: An open source web crawling and web scraping framework for Python," [Online]. Available: https://scrapy.org/, 2026.